

PKCERT AI & Software Development Internship

Task 24 – Convolutional Neural Networks & Transfer Learning: CIFAR-10, Custom Datasets, and Pretrained Architectures

Objective:

Develop a rigorous understanding of convolutional neural networks (CNNs) for image classification, spanning custom architecture design, training from scratch, and transfer learning with modern pretrained backbones. Interns will build and train a CNN on CIFAR-10 or a self-selected custom dataset, then leverage pretrained models (ResNet, VGG, MobileNet) via feature extraction and fine-tuning, and critically compare the two paradigms across accuracy, training cost, generalization, and deployment feasibility.

Total Marks: 100

Part A – Custom CNN from Scratch (25 Marks)

- Design and implement a CNN architecture from scratch using PyTorch or TensorFlow/Keras, including convolutional, pooling, normalization, and fully connected layers — architecture choices must be justified, not copied from a tutorial verbatim.
- Train the network on CIFAR-10 (or a custom dataset with at least 10 classes and 5,000+ labeled images) with a proper train/validation/test split.
- Implement data augmentation (random crop, horizontal flip, color jitter, cutout/mixup — at least two techniques) and justify their impact on generalization.
- Apply and compare at least two regularization strategies (dropout, weight decay, batch normalization, early stopping) and report their individual effect on validation performance.
- Visualize learned filters from the first convolutional layer and produce feature/activation maps for a sample image at two or more depths of the network; explain what each depth appears to be learning.
- Plot training/validation loss and accuracy curves per epoch, and diagnose whether the model overfits, underfits, or generalizes well, with evidence from the curves.

Part B – Rigorous Training & Evaluation (20 Marks)

- Tune hyperparameters (learning rate, batch size, optimizer choice — SGD with momentum vs. Adam vs. AdamW) using a documented, systematic search (grid, random, or a scheduler-based approach) rather than ad hoc trial and error.
- Implement a learning rate schedule (step decay, cosine annealing, or one-cycle) and show its effect on convergence speed and final accuracy versus a constant learning rate.
- Evaluate the final model using accuracy, macro/micro precision, recall, F1-score, a full confusion matrix, and per-class performance breakdown — explicitly identify and discuss the weakest-performing classes and hypothesize why.
- Run inference-time analysis: report parameter count, FLOPs (or an estimate), and average inference latency per image on CPU and, if available, GPU.

Part C – Transfer Learning with Pretrained Models (30 Marks)

- Load at least three pretrained architectures — ResNet (e.g., ResNet18/50), VGG (e.g., VGG16), and MobileNet (e.g., MobileNetV2/V3) — pretrained on ImageNet, using a standard framework's model zoo.

- For each architecture, implement and compare two transfer learning strategies: (1) feature extraction, where the pretrained backbone is frozen and only a new classification head is trained, and (2) fine-tuning, where some or all backbone layers are unfrozen and trained with a reduced learning rate.
- Correctly handle input preprocessing differences between architectures (input resolution, normalization statistics, channel ordering) and explain why mismatched preprocessing degrades performance.
- Apply discriminative/layer-wise learning rates or a gradual unfreezing strategy during fine-tuning and report its effect compared to fine-tuning all layers at once with a single learning rate.
- For each of the three architectures and both strategies (six configurations total), report accuracy, F1-score, training time, and parameter count trained vs. frozen, and present the results in a single consolidated comparison table.
- Identify the best-performing configuration overall and justify the choice with evidence, considering both performance and computational cost.

Part D – Comparative Analysis & Ablation Study (15 Marks)

- Directly compare the custom CNN from Part A against the best transfer learning configuration from Part C on identical test data — quantify the performance gap and discuss what it reveals about the value of pretrained representations for this dataset size.
- Conduct an ablation study on at least one design choice from the transfer learning setup (e.g., number of unfrozen layers, choice of optimizer, presence/absence of data augmentation during fine-tuning) and report how performance changes as that variable is altered.
- Discuss the trade-offs between the three pretrained architectures (accuracy vs. model size vs. inference speed) and recommend which would be most suitable for (a) a cloud-based service with no latency constraint and (b) an on-device mobile application.
- Include a brief discussion of catastrophic forgetting and negative transfer, and whether any evidence of either was observed in your experiments.

Part E – Documentation & Reflection (10 Marks)

- Save the final trained weights for the custom CNN and the best transfer learning model (state_dict, SavedModel, or equivalent) with clear instructions for reloading and running inference.
- Summarize the full pipeline: architectures used, preprocessing, training strategy, and final results, in a way a new team member could follow without re-reading the code.
- Discuss at least two non-trivial technical challenges encountered (e.g., overfitting on a small custom dataset, mismatched input dimensions, unstable fine-tuning) and how each was diagnosed and resolved.
- Reflect on when transfer learning is preferable to training from scratch, and under what data/compute constraints a custom architecture might still be the better choice.

Assessment Breakdown

Assessment Area	Marks
Custom CNN from Scratch (PyTorch/TensorFlow)	25
Training, Regularization & Evaluation on CIFAR-	20

10/Custom Dataset	
Transfer Learning with Pretrained Models (ResNet, VGG, MobileNet)	30
Comparative Analysis & Ablation Study	15
Documentation & Reflection	10
Total	100

Note: All core model implementation, training, and evaluation must use a standard deep learning framework (PyTorch or TensorFlow/Keras) with its official pretrained model zoo for Part C — reimplementing ResNet/VGG/MobileNet architectures from scratch is not required, but you must be able to explain their architectural design choices (skip connections, VGG's stacked 3x3 convolutions, depthwise separable convolutions) when asked. Use of AutoML or no-code tools for the core modeling is not permitted.